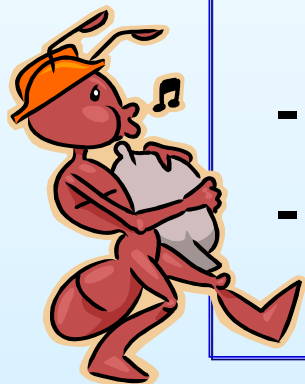


面向切面的软件开发

• 学习目的和要求



---关注点的分离—**理解**

---面向切面的软件开发—**理解**

面向对象编程的模式

面向对象编程 (Object Oriented Programming, OOP) :

- OOP以**对象**为核心。类是对现实世界的抽象，包括表示静态属性的数据和对数据的操作，对象是类的实例化。
- OOP的主要功能是数据**封装**、**继承**和**多态**。
- 面向对象软件工程方法学适用于**大型软件产品**开发。

比如一个业务组件BookService，它有几个业务方法：

- createBook：添加新的Book；
- updateBook：修改Book；
- deleteBook：删除Book。

<https://www.liaoxuefeng.com/wiki/1252599548343744/1266265125480448>

□ 引言

对每个业务方法，例如，`createBook()`，除了业务逻辑，还需要安全检查、日志记录和事务处理。

```
public class BookService {  
    public void createBook(Book book) {  
        securityCheck(); // 安全检查  
        Transaction tx = startTransaction(); // 开始事务  
        try {  
            // 核心业务逻辑，添加书籍到数据库  
            tx.commit(); // 如果业务逻辑执行成功，则提交事务  
        } catch (RuntimeException e) {  
            tx.rollback(); // 如果执行过程中发生异常，则回滚事务  
            throw e; // 将异常向上抛出，以便调用者可以处理  
        }  
        log("created book: " + book); // 记录操作日志  
    }  
}
```

□ 引言

继续编写updateBook(), 代码如下:

```
public class BookService {  
    public void updateBook(Book book) {  
        securityCheck(); // 安全检查  
        Transaction tx = startTransaction(); // 开始事务  
        try {  
            // 核心业务逻辑, 更新数据库中的书籍信息  
            tx.commit(); // 如果业务逻辑执行成功, 则提交事务  
        } catch (RuntimeException e) {  
            tx.rollback(); // 如果业务逻辑执行发生异常, 则回滚事务  
            throw e; // 将异常向上抛出, 以便调用者可以处理  
        }  
        log("updated book: " + book); // 记录操作日志  
    }  
}
```

对于安全检查、事务处理、日志记录等代码, 它们会重复出现在每个业务方法中。

□ 引言

一种可行的方式是使用Proxy模式，将某个功能，例如，权限检查，放入Proxy中：

```
// SecurityCheckBookService 类作为代理类,实现了 BookService 接口。  
public class SecurityCheckBookService implements BookService  
{  
    private final BookService target; // target 指向执行业务操作的目标对象  
    public SecurityCheckBookService(BookService target) {  
        this.target = target;  
    }  
    public void createBook(Book book) {  
        securityCheck(); // 安全检查  
        target.createBook(book); //调用目标对象的 createBook 方法  
    }  
}
```

□ 引言

一种可行的方式是使用Proxy模式，将某个功能，例如，权限检查，放入Proxy中：

```
public void updateBook(Book book) {  
    securityCheck();  
    target.updateBook(book);  
}  
  
public void deleteBook(Book book) {  
    securityCheck();  
    target.deleteBook(book);  
}  
  
private void securityCheck() {
```

这种方式的缺点是比较麻烦，必须先抽取接口，然后，针对每个方法实现Proxy。

□ 引言

另一种方法是，既然SecurityCheckBookService的代码都是标准的Proxy样板代码，不如把权限检查视作一种切面（Aspect），把日志、事务也视为切面，然后，以某种自动化的方式，把切面织入到核心逻辑中，实现Proxy模式。

以AOP的视角来编写上述业务，可以依次实现：

- 核心逻辑，即BookService；
- 切面逻辑，即：
 - 权限检查的Aspect；
 - 日志的Aspect；
 - 事务的Aspect。

然后，以某种方式，让框架来把上述3个Aspect以Proxy的方式“织入”到BookService中，这样一来，就不必编写复杂而冗长的Proxy模式。

<https://www.liaoxuefeng.com/wiki/1252599548343744/1266265125480448>

一、面向切面编程

1、面向切面编程概述

面向切面编程（AOP）旨在增加程序的模块化，通过将**横切关注点**（如日志记录、事务管理、安全检查等）与业务逻辑分离，来提高代码的可维护性和重用性。

□ 面向切面编程

- 提供一种与主业务逻辑分离的方式来处理程序中的横切关注点，从而使得这些关注点能够独立于主业务逻辑进行管理和维护。
- 在现有代码程序中，在程序生命周期或者横向流程中“加入/减去”一个或多个功能，不影响原有功能。

□ 横切关注点

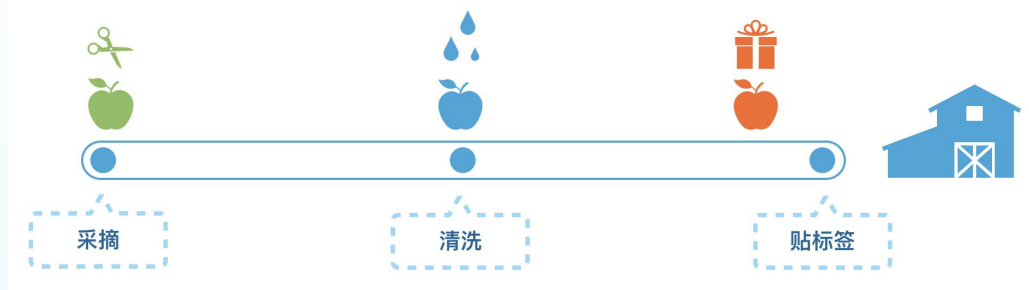
- **横切关注点**是指那些与应用程序的核心业务逻辑无关的功能，如日志记录、事务管理、权限控制等。
- AOP可以将这些横切关注点与核心业务逻辑分离，以便更好地实现模块化、复用和清晰的代码结构。

一、面向切面编程

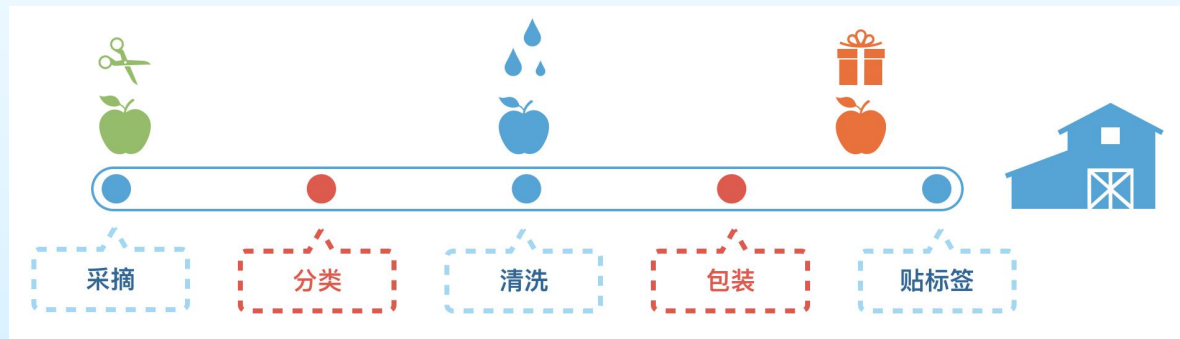
1、面向切面编程概述

生活中的例子

- 农场的水果包装流水线一开始只有“采摘-清洗-贴标签”



- 为了提高销量，想加上两道工序“分类和包装”，但又不能干扰原有的流程，同时如果没增加收益可以随时撤销新增工序。

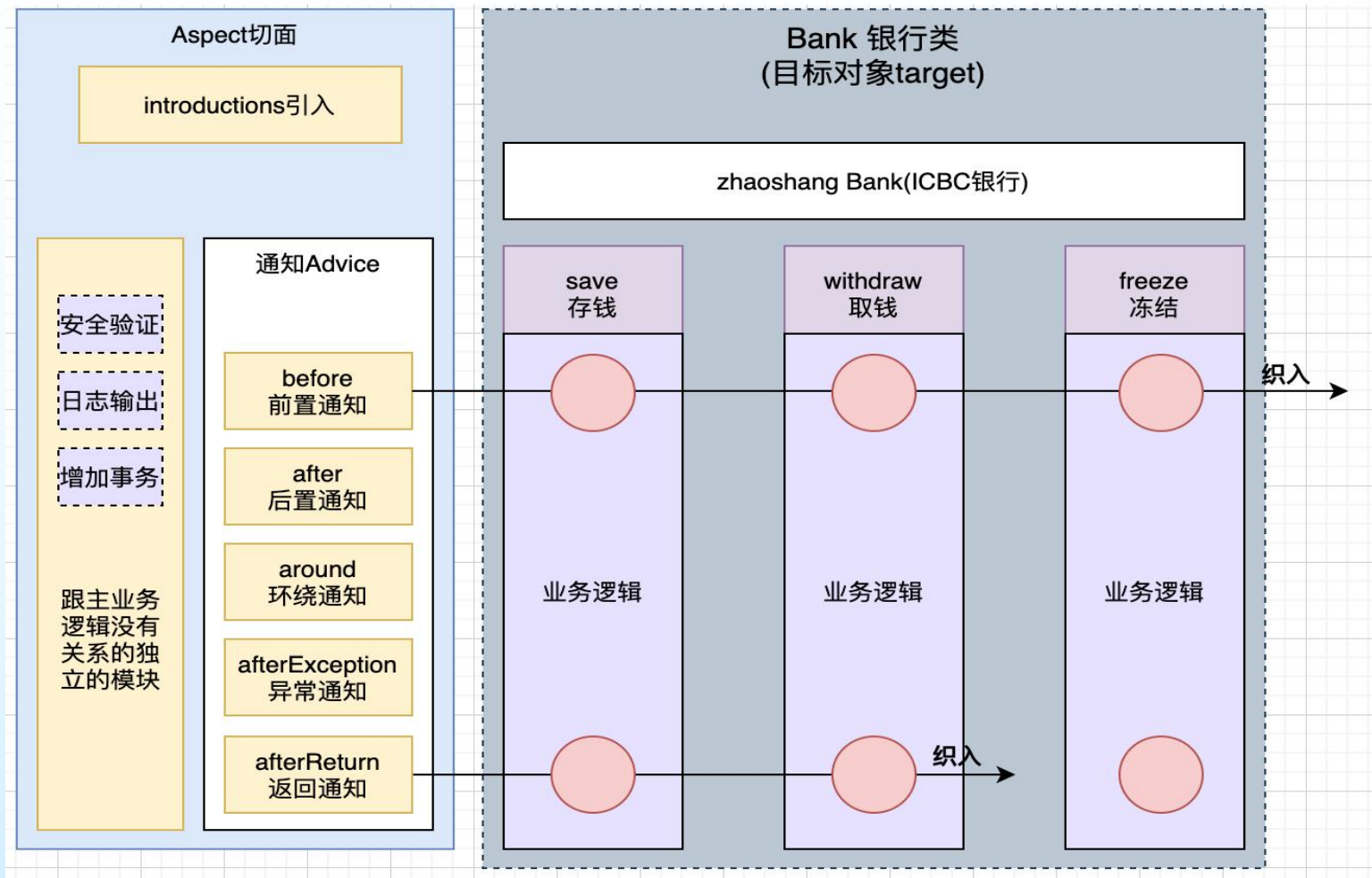


- 最后在流水线的中的空隙插上两个工人去处理，形成“采摘-分类-清洗-包装-贴标签”的新流程，而且工人可以随时撤回。

一、面向切面编程

1、面向切面编程概述

生活中的例子



一、面向切面编程

(1) 切面 (Aspect)

切面定义了横切关注点的实现，并且指定了这些关注点在何时、何地 with 主业务逻辑相结合。

□ 在AOP中，最核心的概念是切面 (Aspect)

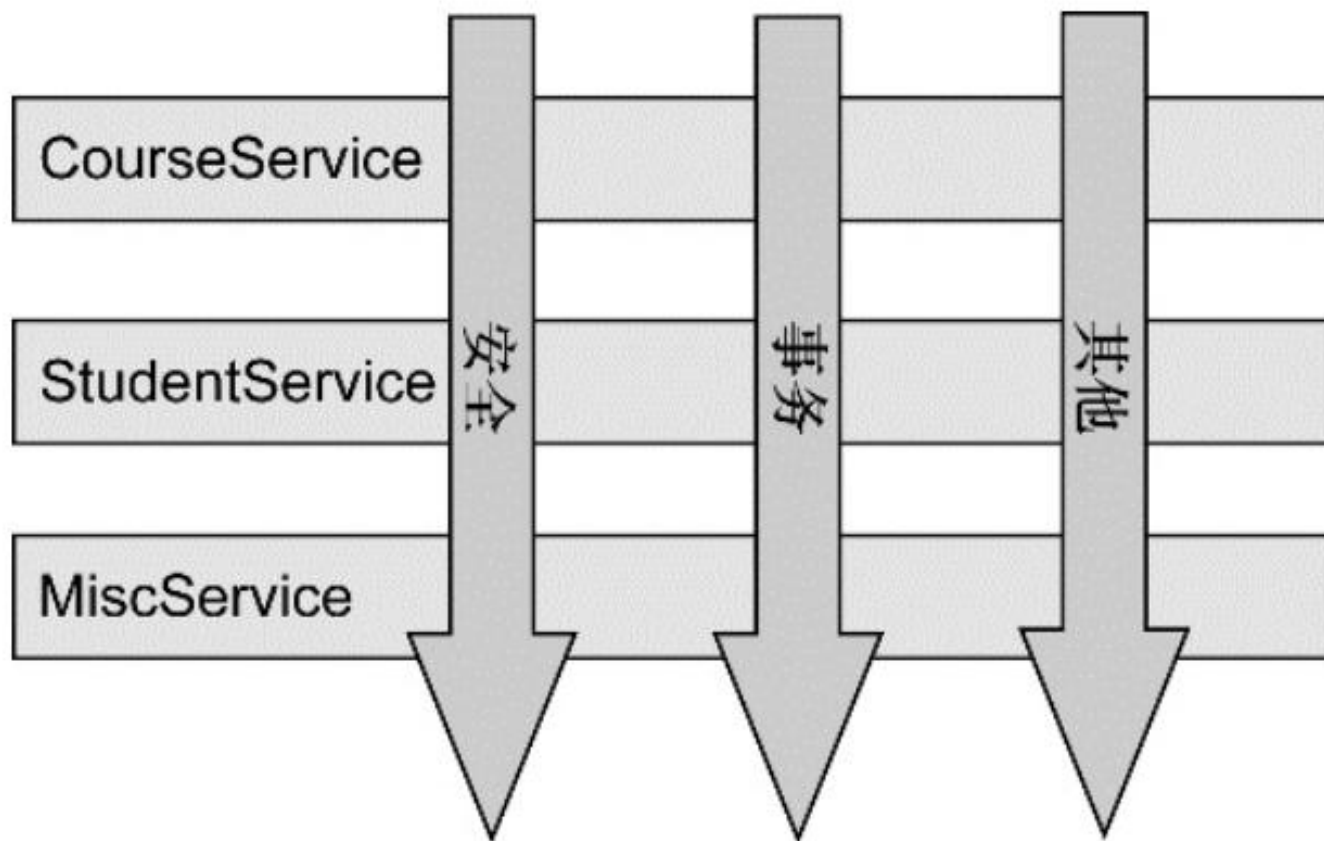
- 切面将那些影响多个类的行为（横切关注点）集中起来描述的模块化单元。例如，日志记录可能会散布在多个类中，但在AOP中，它们被一个切面所封装，从而实现关注点的分离。

□ 切面通过通知 (Advice) 和切点 (Pointcut) 来具体实现。

- 通知定义了切面是如何增强目标对象的行为的。它指定了切面的代码在目标对象的方法执行之前、之后或者抛出异常时执行。
- 切点定义了通知应当应用的位置，即在哪些目标对象的哪些方法上执行切面代码。通过精确指定切点，可以精确控制代码的横切逻辑。

一、面向切面编程

(1) 切面 (Aspect)



切面实现了横切关注点(跨多个应用对象的逻辑)的模块化

一、面向切面编程

(1) 切面 (Aspect)

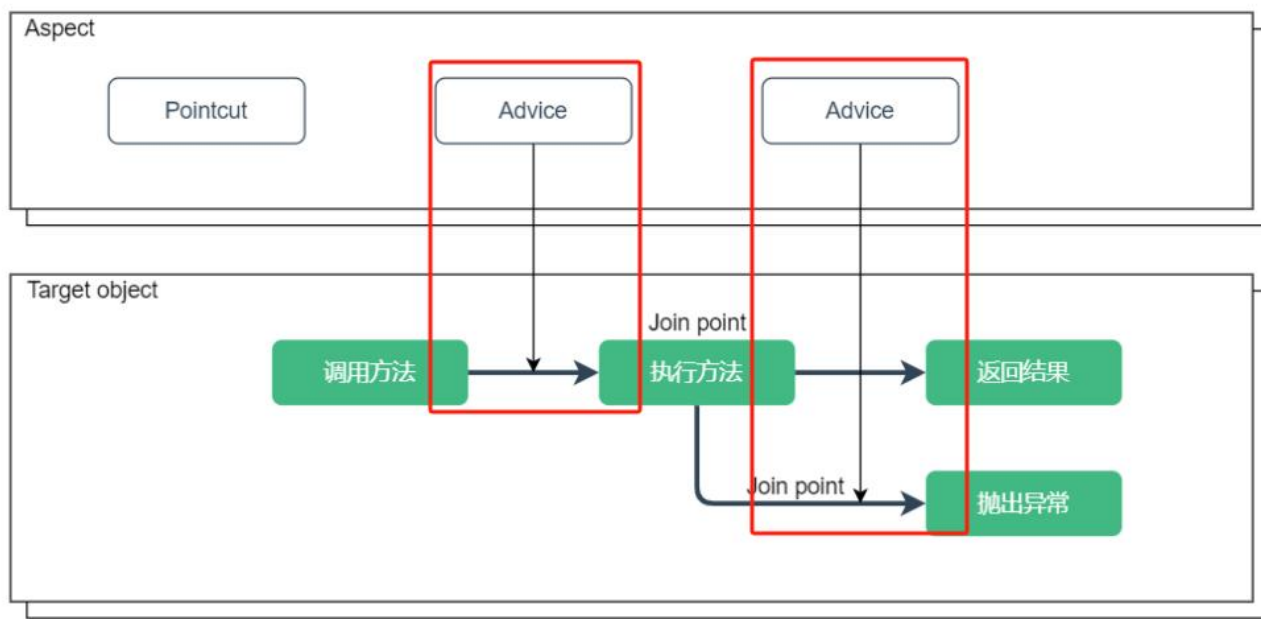
在这个例子中，通过使用**@Aspect**注解来标识这是一个切面类，并通过**@Component**注解将其注册为Spring组件。

```
@Component  
@Aspect  
@Order(1)  
public class SystemLogAspect {  
    //切面逻辑...  
}
```

一、面向切面编程

(2) 通知(Advice)

通知定义了切面的具体功能是什么以及何时被使用，并指定了目标方法执行前、执行后或抛出异常时应该执行的逻辑。



一、面向切面编程

(2) 通知(Advice)

在AOP中，根据使用时机的不同，通知被分为下面5种类型：

- 前置通知(Before)：在目标方法被调用之前调用的通知功能。
- 后置通知(After)：在目标方法完成后调用的通知功能，即使在执行目标方法出现异常，也照常执行。
- 返回通知(After-returning)：在目标方法成功执行之后才调用通知功能。
- 异常通知(After-throwing)：在目标方法抛出异常后执行通知功能。
- 环绕通知(Around)：通知包围了目标方法，在调用目标方法前后可以执行自定义的通知功能。

一、面向切面编程

(2) 通知(Advice)

通知注解使用在切面类中的方法上，有前置通知**@Before**、
后置返回通知**@AfterReturning**、后置异常通知
@AfterThrowing、后置最终通知**@After**、环绕通知**@Around**

```
@Component  
@Aspect  
@Order(1)  
public class SystemLogAspect {  
    public SystemLogAspect(){  
        @Around("execution(* com.inphase..*.*(..)) &&  
@annotation(systemLogAnnotation)")  
        public Object aroundMethod(){  
    }  
}
```

一、面向切面编程

(3) 切入点 (Pointcut)

切入点是一组连接点的集合，定义了通知应该在哪些连接点上执行。

□ 在AOP中，切入点将通知“织入”到连接点中

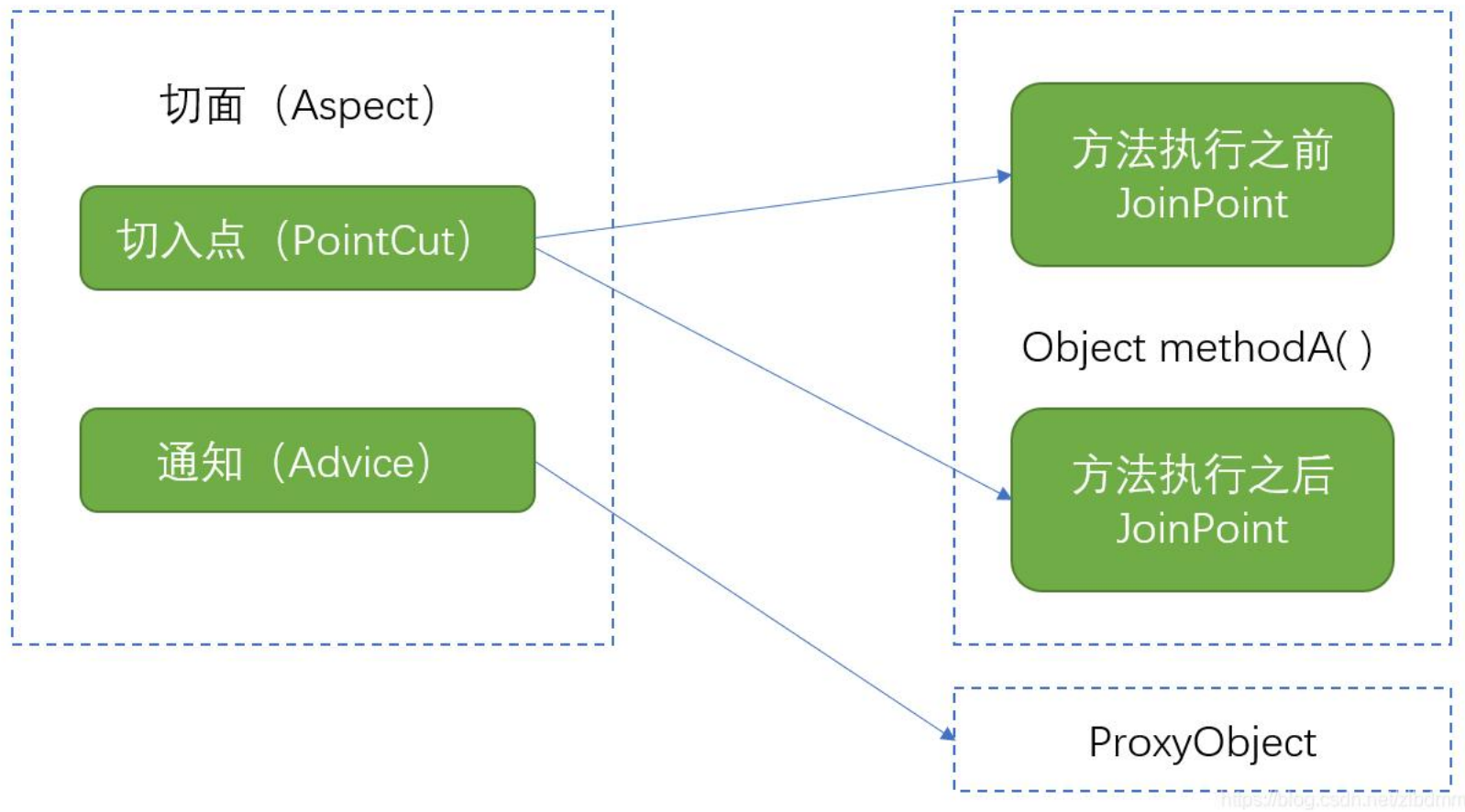
➤ 通过配置切点，可以将通知“织入”到一个或者多个连接点中。

□ 切入点通过指定类名称、方法名称或正则表达式来匹配

➤ 在实际应用中，是通过指定类名称、方法名称、正则表达式来进行来匹配指定切点，在AOP中，使用AspectJ提供的切入点表达式语言来定义切入点。

一、面向切面编程

(3) 切入点 (Pointcut)



一、面向切面编程

(3) 切入点 (Pointcut)

```
@Aspect
public class MyAspect{
    @Pointcut(value="execution(* com.inphase..*.*(..)) &&
@annotation(com.inphase.mj.industry.common.log.annotation.
SystemLogAnnotation)")
    public void myPointcut() {}
    @Around(value="myPointCut()")
    public Object aroundMethod(PropceedingJoinPoint
joinPoint) throws Throwable{
        return null;
    }
}
```

在这个例子中，使用**@Pointcut**注解来定义一个切点，并通过
"execution(* com.inphase..(..))"来指定要匹配的方法。

一、面向切面编程

(4) 连接点 (Join Point)

连接点是在应用程序中你想要插入切面的点。在Spring AOP中，连接点总是表示一个方法的执行。

□ 连接点是在业务逻辑执行过程中能够插入切面通知的一个点。

- 在Spring中，连接点包括在调用方法时、调用方法后、或在抛出异常时。切面代码可以利用这些连接点插入到应用程序的正常流程，并引入新的行为。
- Spring AOP是基于动态代理的，所以它只支持方法的连接点，但像其他的AOP框架如AspectJ在修改属性时或者构造器执行时都可以织入通知，达到更细颗粒度的控制拦截。

一、面向切面编程

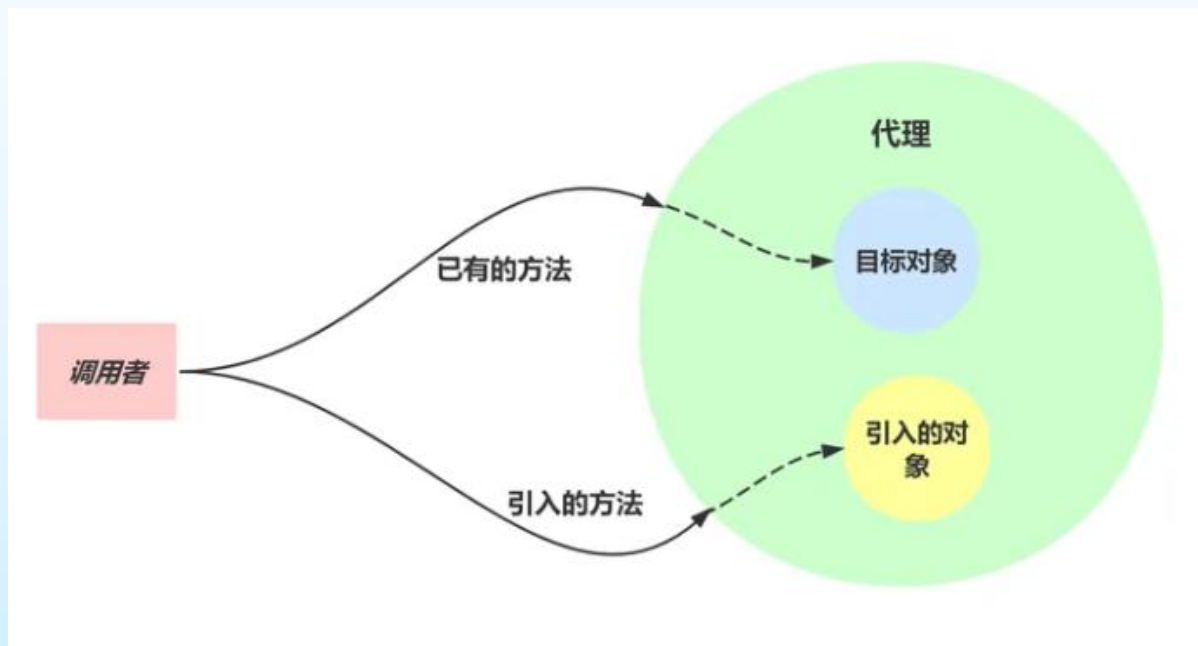
(4) 连接点 (Join Point)

```
@Aspect
public class MyAspect{
    @Pointcut(value="execution(* com.inphase..*.*(..)) &&
@annotation(com.inphase.mj.industry.common.log.annotation.
    SystemLogAnnotation)")
    public void myPointcut() {}
    @Around(value="myPointCut()")
    public Object aroundMethod(JoinPoint JoinPoint) throws
    Throwable{
        jointPoint.getSignature().getName();
        return null;
    }
}
```

一、面向切面编程

(5) 引入 (Introduction)

允许向类中添加新的属性和方法, 实际上就是增强类(让类拥有原来没有的功能或者属性)。具体可以理解成将切面应用到目标类上, 从而实现不修改类的代码, 而让它具有新的功能和属性。



一、面向切面编程

(5) 引入 (Introduction)

```
@Aspect // 标识其为一个AOP切面
@Component
public class SysLogServiceIntroducer {
    @DeclareParents(value =
        "com.aaron.SpringBoot1.service.AnimalService",
        defaultImpl = SysLogServiceImpl.class)
    public static SysLogService sysLogService;
}
```

一、面向切面编程

(5) 引入 (Introduction)

```
@Aspect // 标识其为一个AOP切面  
@Component  
public class SysLogServiceIntro  
    @DeclareParents(value =  
        "com.aaron.SpringBoot1.service  
        defaultImpl = SysLo  
    public static SysLogService s  
}
```

```
// 引入的新接口  
public interface SysLogService {  
    String getSysMsg();  
}  
// 引入的新接口的实现类  
...  
@Service  
public class SysLogServiceImpl  
    implements SysLogService {  
    @Override  
    public String getSysMsg() {  
        String msg = "This is a method  
        introduction by AOP";  
        System.out.println(msg);  
        return msg;  
    }  
}
```

一、面向切面编程

(5) 引入

```
@Aspect // 标识  
@Component  
public class Sys  
@DeclareParent  
"com.aaron.Spring"  
default  
public static S  
}
```

```
@Controller  
@RequestMapping(onConstructor =  
@_(@Autowired))  
@RequestMapping("animal")  
public class AnimalController {  
  
    private final AnimalService animalService;  
  
    @RequestMapping("/test3")  
    @ResponseBody  
    public String test3() {  
        // 调用animalService方法  
        animalService.test1();  
        // 调用animalService中通过AOP引入的新方法  
        if(animalService instanceof SysLogService) {  
            ((SysLogService)animalService).getSysMsg();  
        }  
        return "OK";  
    }  
}
```

```
service {
```

```
celImpl  
ce {
```

```
g() {  
s a method  
sg);
```

一、面向切面编程

(6) 织入 (Weaving)

织入是把切面应用到目标对象并创建新的代理对象的过程，这是一个动态的过程，切面在指定的切点被织入到目标对象中，实际上在目标对象的生命周期中，存在以下的多个点可以进行织入：

□ 编译期

- 切面在目标类编译时被织入。这种方式需要特殊的编译器。AspectJ 的织入编译器就是以这种方式织入切面的。

□ 类加载期

- 切面在目标类加载到 JVM 时被织入。这种方式需要特殊的类加载器 (ClassLoader)，它可以在目标类被引入应用之前增强该目标类的字节码。AspectJ 5 的加载时织入 (load-time weaving, LTW) 就支持以这种方式织入切面。

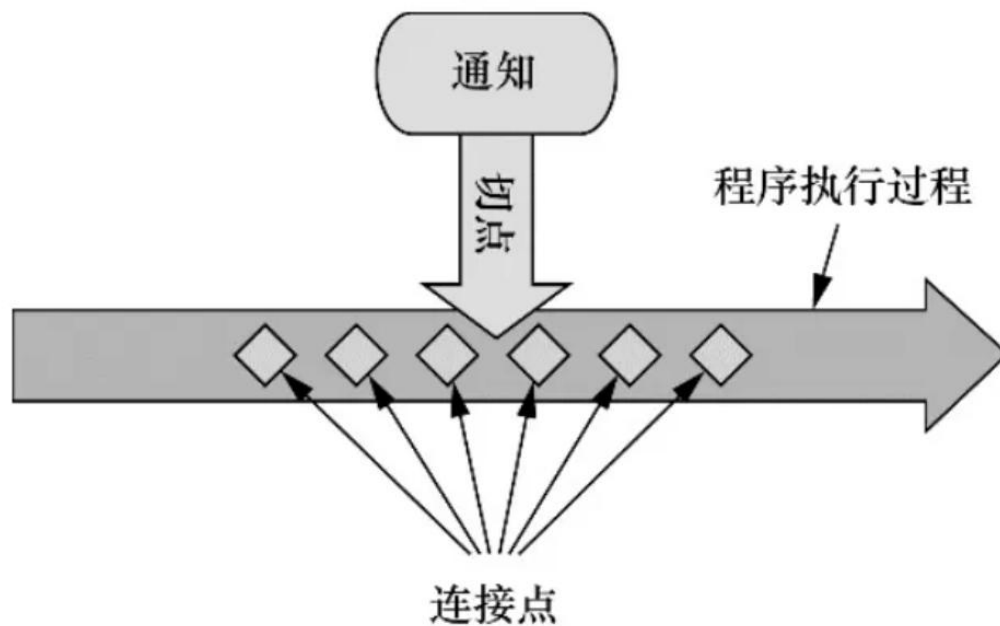
□ 运行期

- 切面在应用运行的某个时刻被织入。一般情况下，在织入切面时，AOP 容器会为目标对象动态地创建一个代理对象。Spring AOP 就是以这种方式织入切面的。

一、面向切面编程

(6) 织入 (Weaving)

如图所示，在一个或多个连接点上，可以把切面的功能（通知）织入到程序的执行过程中。



一、面向切面编程

(7) 代理 (Proxy)

□ 代理

- 可以理解成委托别人帮忙处理事情，生活中最常见的例子就是中介。真正房东一般会给中介佣金然后将房租的出租等事情委托给中介帮忙处理，在这个例子中介就是房东的代理，负责帮房东跑腿。

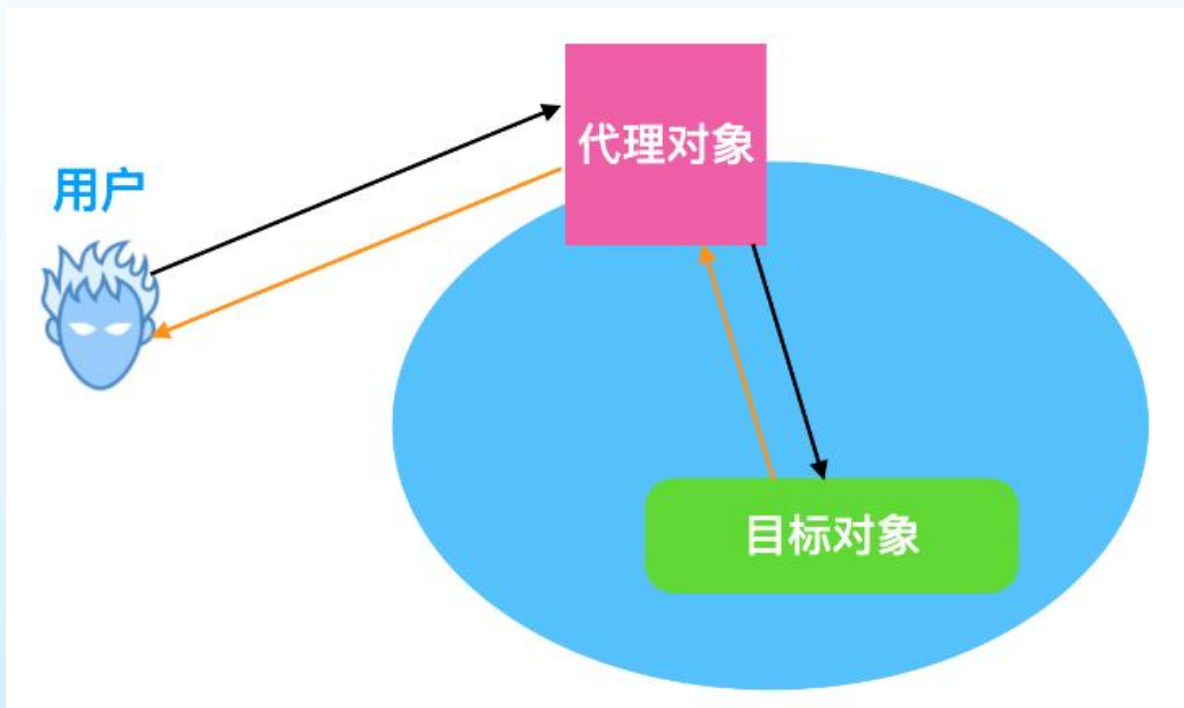
□ 动态代理

- 在程序运行期, 动态创建目标对象的代理对象, 并对目标对象中的方法进行功能性增强的一种技术, 根据实现方式可以划分为: JDK动态代理和CGlib动态代理。

一、面向切面编程

(7) 代理 (Proxy)

如图所示，用户不能直接使用目标对象，而是构造出一个代理对象，由代理对象作为中转，代理对象负责调用目标对象真正的行为，从而把结果返回给用户。也就是说，代理的关键点就是代理对象和目标对象的关系。

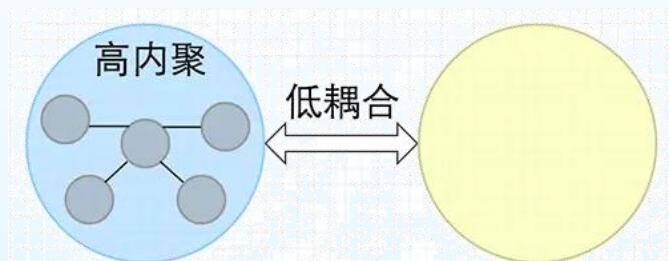


一、面向切面编程

3、面向切面编程的作用

□ 降低模块间的耦合度

- AOP技术的出现，将系统的核心关注点和横切关注点分离，避免了非核心业务耦合在核心业务中，降低了模块间的耦合度，提高了系统的可读性、可操作性和可维护性。

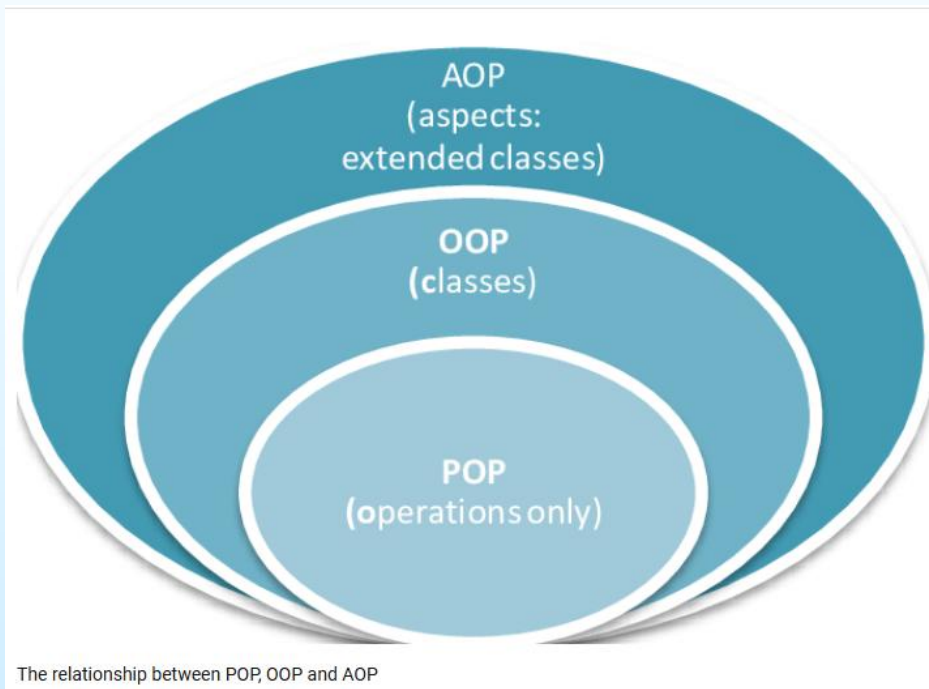


□ 代码复用

- AOP技术将与核心业务无关或者关系不大却为不同业务模块公用或者需要的逻辑抽取成新的模块，在需要的地方再引入，大大减少的系统的重复代码。

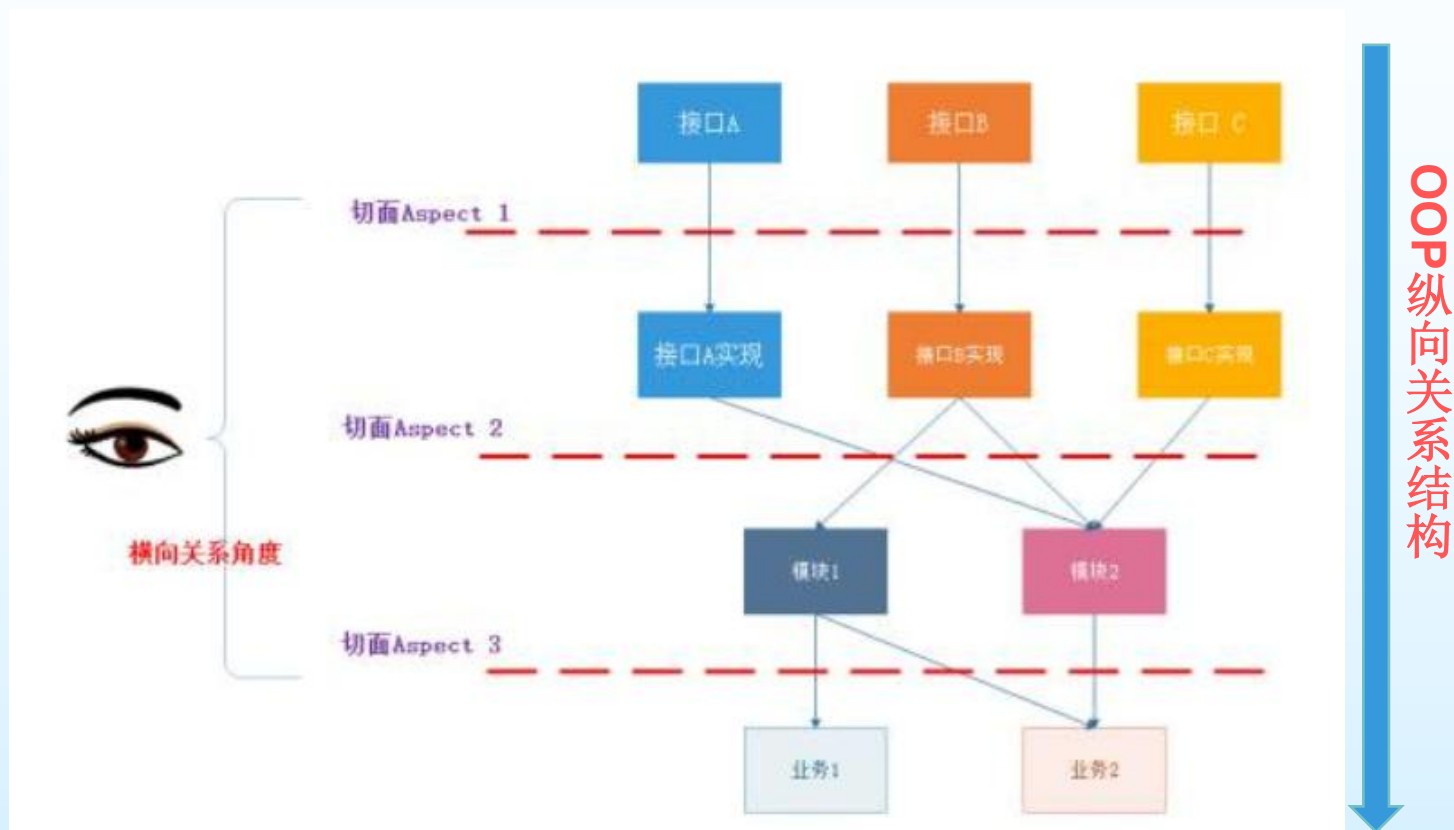
二、AOP与OOP之间的关系

- AOP和OOP（面向对象编程）是两种不同的编程思想。OOP将程序各个功能模块封装成对象，通过对象间的交互来实现系统功能，强调代码的结构和组织。而AOP则是在OOP的基础上，通过切面的方式来处理横切关注点，强调代码的模块化和复用。
- AOP和OOP可以相互补充和结合使用，AOP可以用于解决OOP难以处理的横切关注点问题，例如日志记录、性能监控等；而OOP可以用于实现具体的业务逻辑和对象间的交互。通过结合使用AOP和OOP，可以实现更加灵活和可维护的软件系统。



二、AOP与OOP之间的关系

横向！ AOP开发最重要的指导思想就是**角度**，而最重要的**观察角度就是横向**（如图所示的程序员的眼睛那样观察）。



三、面向切面编程的实现技术

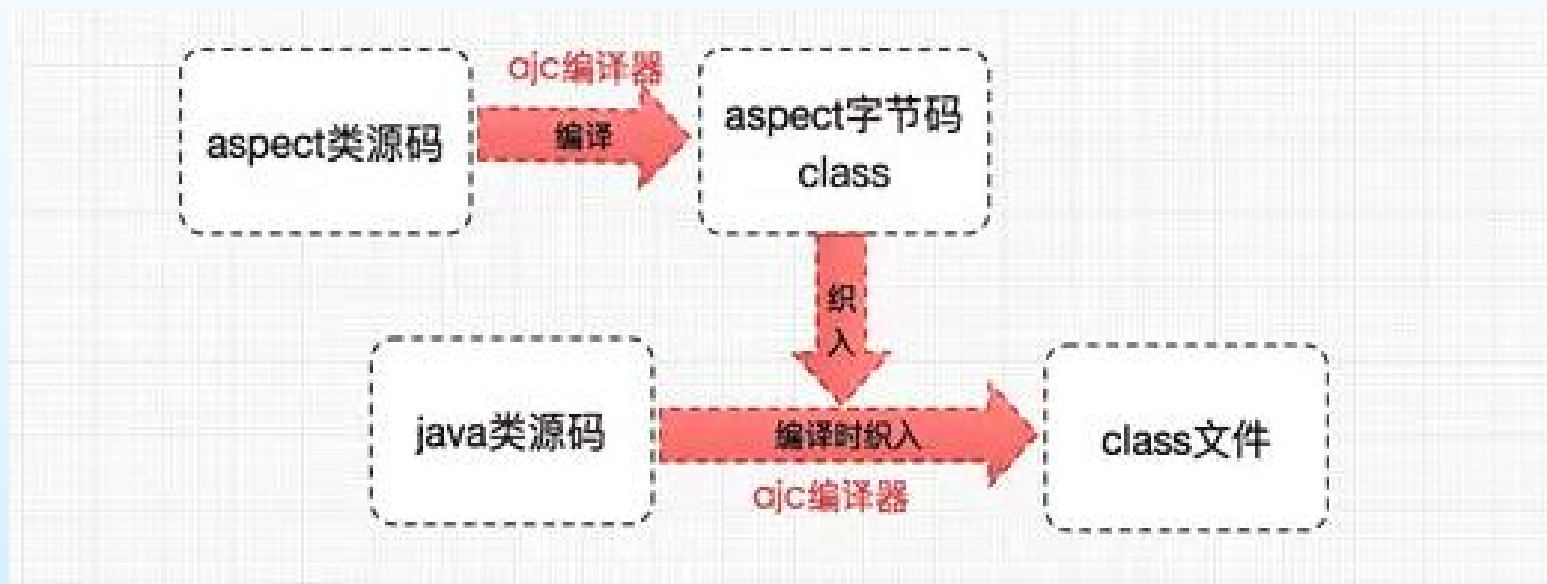
(1) 静态AOP实现

采用了这种方案的AOP框架会在编译阶段对程序**需要增强的代码进行修改**，从而生成静态的AOP代理类，即这样实际上编译出来的类的class文件实际上已经是增强后的代理类，因为编译期的特殊性，所以要实现静态AOP需要用特殊的编译器，**常见的静态AOP框架如：AspectJ。**

三、面向切面编程的实现技术

AspectJ应用到java代码的过程（这个过程称为**织入**），对于织入这个概念，可以简单理解为**aspect (切面) 应用到目标函数 (类)**的过程。

AspectJ采用的就是**静态织入**的方式。AspectJ主要采用的是**编译期织入**，在这个期间使用AspectJ的ajc编译器(类似javac)把aspect类编译成class字节码后，在java目标类编译时织入，即先编译aspect类再编译目标类。



编译期织入

三、面向切面编程的实现技术

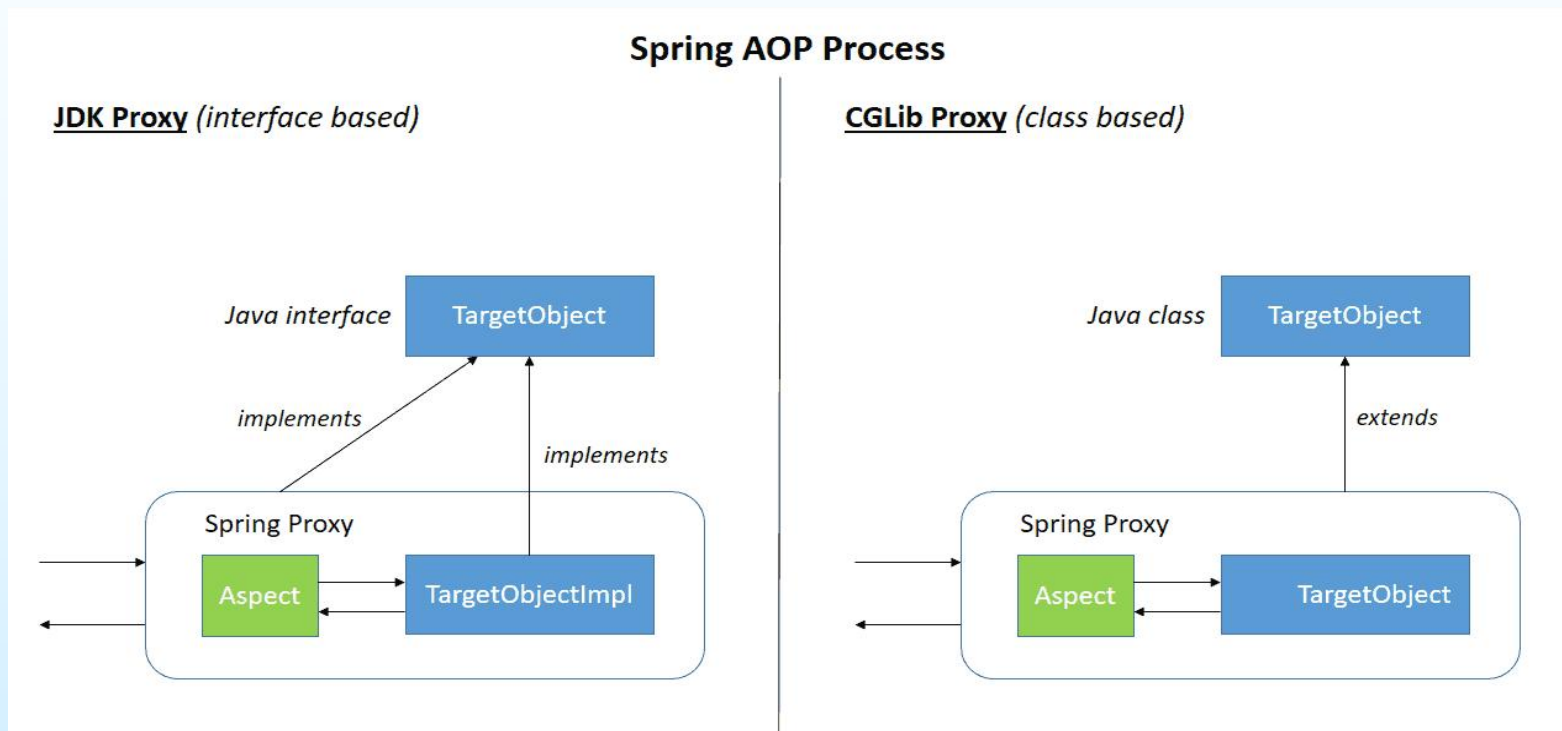
(2) 动态AOP实现

采用这种方案的AOP框架是在运行时才会对目标对象生成代理对象(如在内存: 通过JDK的动态代理技术或者CGlib动态字节码技术生成AOP代理对象), 这种方案常见的框架: Spring AOP。

三、面向切面编程的实现技术

JDK动态代理需要被代理的类必须实现一个**接口**，底层通过反射实现。

CGLIB动态代理可以不用需要被代理类必须实现接口，底层通过**继承**实现，因此如果某个类被标记为**final**，那么它是无法使用**CGLIB**做动态代理的。



三、面向切面编程的实现技术

(3) 不同AOP框架之间的特点比较

类别	实现	原理	优点	缺点
静态 AOP	在编译期，在 class 文件实现增强	在编译期，通知直接以字节码的形式织入到目标类的字节码 (.class) 文件中，实现增强	增强类性能不受影响(因为在编译期就已经增强了，还没有到运行时)	1、不够灵活、需要特殊的编译器 举例： AspectJ 框架
动态 AOP	JDK 的动态代理	在运行时，通过反射为目标类动态生成代理类，并将通知织入到代理类中	1、相比静态 AOP 更加灵活 2、JDK 原生支持，使用更加简单	1、必须实现接口(因为生成的代理都会继承 Proxy 类，所以使用该种方式必须实现接口) 2、因为是在运行时通过反射动态生成代理类，对系统的性能会存在一定影响

三、面向切面编程的实现技术

(3) 不同AOP框架之间的特点比较

类别	实现	原理	优点	缺点
动态字节码生成	CGLIB	在运行时，目标类加载完成后，利用ASM- 开源的 Java 字节码编辑库，动态生成目标类的子类代理对象，并将通知织入到代理对象中	无需实现接口，更加灵活	1、在运行时动态生成代理对象，对系统性能存在一定影响 2、对类存在更多限制，因为最后是通过修改目标类的字节码生成对应子类的代理对象，所以要求目标类不能被final 修饰
自定义类加载器	可以考虑 <u>javassist</u> 来实现 原理是在字节码加载前直接修改需要切入的方法。	在运行前，目标加载前，将切面逻辑加到目标字节码中。	可以对绝大部分类织入。	如果用到了其他类加载器，则这些类将不被织入

四、AOP在软件开发中的应用

(1) 事务管理 (TRANSACTION MANAGEMENT)

在企业级应用中，事务管理是一个典型的横切关注点。利用**AOP**，可以将事务管理逻辑从业务逻辑中抽离，使业务逻辑更加清晰。

(2) 日志记录 (LOGGING)

通过使用**AOP**实现日志记录，可以将日志记录逻辑集中到一个切面中，而不是分散在多个业务逻辑类中。

(3) 性能监控 (PERFORMANCE MONITORING)

AOP也常用于实现性能监控功能，通过切面代码监控方法执行的时间，来帮助优化程序性能。

谢谢观看

